



Security Operations Center (SOC) Simulation Project Report

Submitted By: Vishal Singh Ohlan



1. Introduction

Overview

In the modern cybersecurity landscape, the ability to detect, analyze, and respond to threats in real-time is a critical requirement for any resilient infrastructure. This project documents the end-to-end implementation of a localized **Security Operations Center (SOC)** environment. The primary objective was to establish a centralized monitoring system capable of ingesting host telemetry, identifying malicious patterns, and facilitating forensic investigations.

Scope of Work

The project was executed in a controlled virtual environment utilizing a **VMware-based Ubuntu 20.04 workstation**. The methodology followed the standard incident response lifecycle, encompassing:

- **Infrastructure Deployment:** Setting up the ELK (Elasticsearch, Logstash, Kibana) stack for centralized logging.
- **Detection Engineering:** Developing custom SIEM rules to identify unauthorized access attempts.
- **Attack Simulation:** Executing manual brute-force scenarios to validate defensive alerting.
- **Vulnerability Management:** Identifying system-level risks using industry-standard scanning tools.
- **Endpoint Forensics:** Utilizing SQL-based auditing and database analysis to reconstruct user activity.

Core Toolset

To simulate a professional security environment, the following tools were integrated:

- **SIEM:** Elastic Stack (Elasticsearch & Kibana) for data indexing and visualization.
- **Vulnerability Scanner:** Nessus Essentials for automated risk assessment.
- **Live Response:** Osquery for real-time endpoint auditing.
- **Forensics:** SQLite3 for browser history and artifact analysis.



Project Goal

The ultimate goal of this project was to bridge the gap between theoretical security concepts and practical application. By building this lab, we demonstrate the transition from **passive logging** to **active threat hunting**, ensuring that security incidents are not just recorded but analyzed and remediated effectively.

2. Methodology & Core Concepts

2.1. SOC Operational Workflow

The project was conducted following the standard functional roles and workflows of a modern Security Operations Center. The simulation mirrors the transition of an event through the Analyst Tiers:

- Detection & Triage (Tier 1): Monitoring the SIEM for initial alerts.
- Deep Investigation (Tier 2): Utilizing forensics (Osquery/SQLite) to determine the scope of a breach.
- Remediation (Tier 3): Identifying root causes through vulnerability management.

2.2. The CIA Triad & Risk Management

All defensive measures implemented in this lab were designed to uphold the CIA Triad:

- Confidentiality: Ensuring only authorized users can access the workstation.
- Integrity: Using logs to verify that system files remain unaltered.
- Availability: Monitoring service health to ensure the SIEM remains operational.

Within this context, we identified Vulnerabilities (unpatched software), mapped potential Threats (Brute Force attempts), and calculated the resulting Risk to the infrastructure.

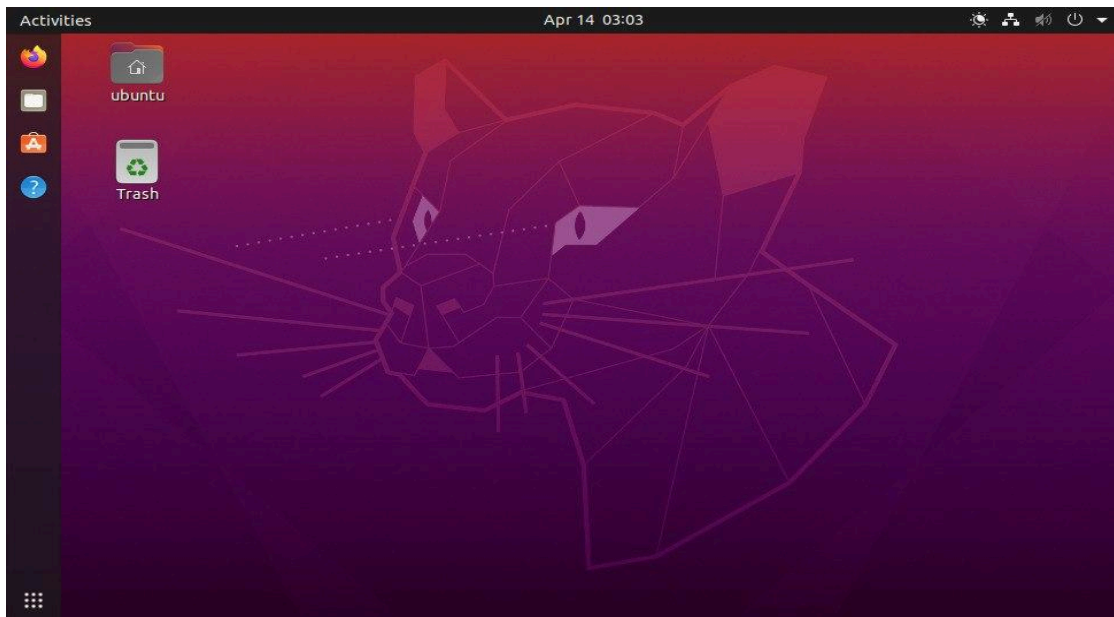
3. Lab Setup

3.1. Environment Setup

The lab environment was created using VMware Workstation with Ubuntu as the SOC machine. Elastic Stack (Elasticsearch and Kibana) was used for log monitoring and visualization.



- **System Initialization:** The Ubuntu virtual machine was started.



- **SIEM Core Service Validation:** The health of the *ELK Stack* (Elasticsearch & Kibana) was audited. Using **systemctl**, we confirmed that the Elasticsearch engine—responsible for data indexing—and Kibana—the visualization layer—were both in an **active (running)** state. This step is vital to ensure that the detection pipeline does not face downtime during active simulations.

```
root@ubuntu:/home/ubuntu# sudo systemctl status elasticsearch
● elasticsearch.service - Elasticsearch
   Loaded: loaded (/lib/systemd/system/elasticsearch.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2026-04-14 03:17:20 PDT; 10min ago
     Docs: https://www.elastic.co
   Main PID: 955 (java)
    Tasks: 102 (limit: 9365)
   Memory: 4.1G
   CGroup: /system.slice/elasticsearch.service
           └─ 955 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.name=serv
              1201 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -Des.networkad
              1657 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux-x86_64/bin/controller

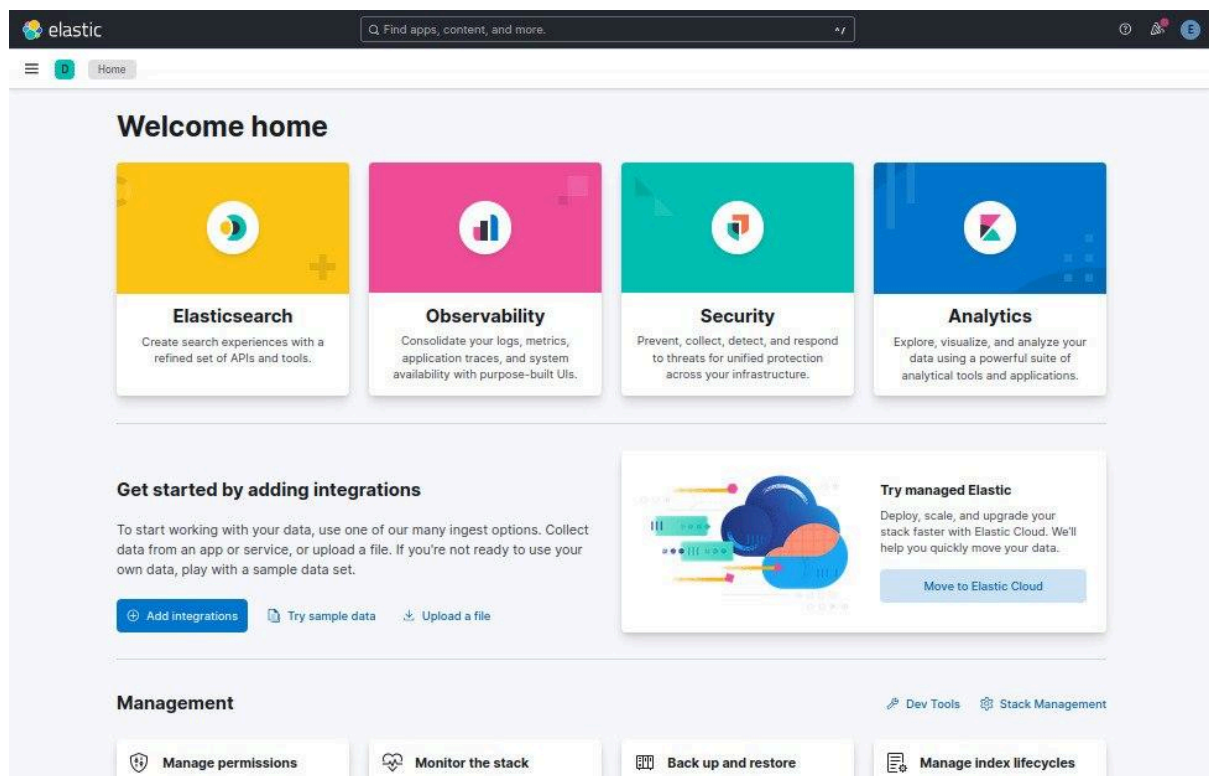
Apr 14 03:00:22 ubuntu systemd[1]: Starting Elasticsearch...
Apr 14 03:17:20 ubuntu systemd[1]: Started Elasticsearch.

root@ubuntu:/home/ubuntu# sudo systemctl status kibana
● kibana.service - Kibana
   Loaded: loaded (/lib/systemd/system/kibana.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2026-04-14 03:00:22 PDT; 28min ago
     Docs: https://www.elastic.co
   Main PID: 957 (node)
    Tasks: 11 (limit: 9365)
   Memory: 627.2M
   CGroup: /system.slice/kibana.service
           └─ 957 /usr/share/kibana/bin/./node/glibc-217/bin/node /usr/share/kibana/bin/./src/cli/kiba

Apr 14 03:00:22 ubuntu systemd[1]: Started Kibana.
Apr 14 03:00:40 ubuntu kibana[957]: Kibana is currently running with legacy OpenSSL providers enabled! Fo
Apr 14 03:01:37 ubuntu kibana[957]: {"log.level":"info","@timestamp":"2026-04-14T10:01:37.653Z","log.logg
Apr 14 03:01:45 ubuntu kibana[957]: Native global console methods have been overridden in production envi
Apr 14 03:03:19 ubuntu kibana[957]: [2026-04-14T03:03:18.020-07:00][INFO ][root] Kibana is starting
Apr 14 03:03:19 ubuntu kibana[957]: [2026-04-14T03:03:19.360-07:00][INFO ][node] Kibana process configure
Apr 14 03:18:27 ubuntu kibana[957]: [2026-04-14T03:18:26.702-07:00][INFO ][plugins-service] The following
Apr 14 03:18:27 ubuntu kibana[957]: [2026-04-14T03:18:27.856-07:00][INFO ][http-server] Prebootl http serv
```



- **SOC Management Interface Access:** Following service confirmation, the web-based management console was accessed. The **Kibana “Welcome Home”** dashboard confirmed that the Security and Analytics modules were properly initialized. This interface acts as the "Single Pane of Glass" for the SOC analyst, providing access to rule management, telemetry discovery, and incident response tools.



4. Log Collection

To test the log flow, the **tail** command was used to monitor **/var/log/syslog** in real-time. This verified that the system was capturing raw authentication events.

- **Manual Validation:** A "Test message 1" was injected via the command line to verify immediate log capture.
- **Initial Discovery:** We observed several "Failed login attempt" entries originating from local IP ranges. This raw data confirms that the host is a viable source for security monitoring.

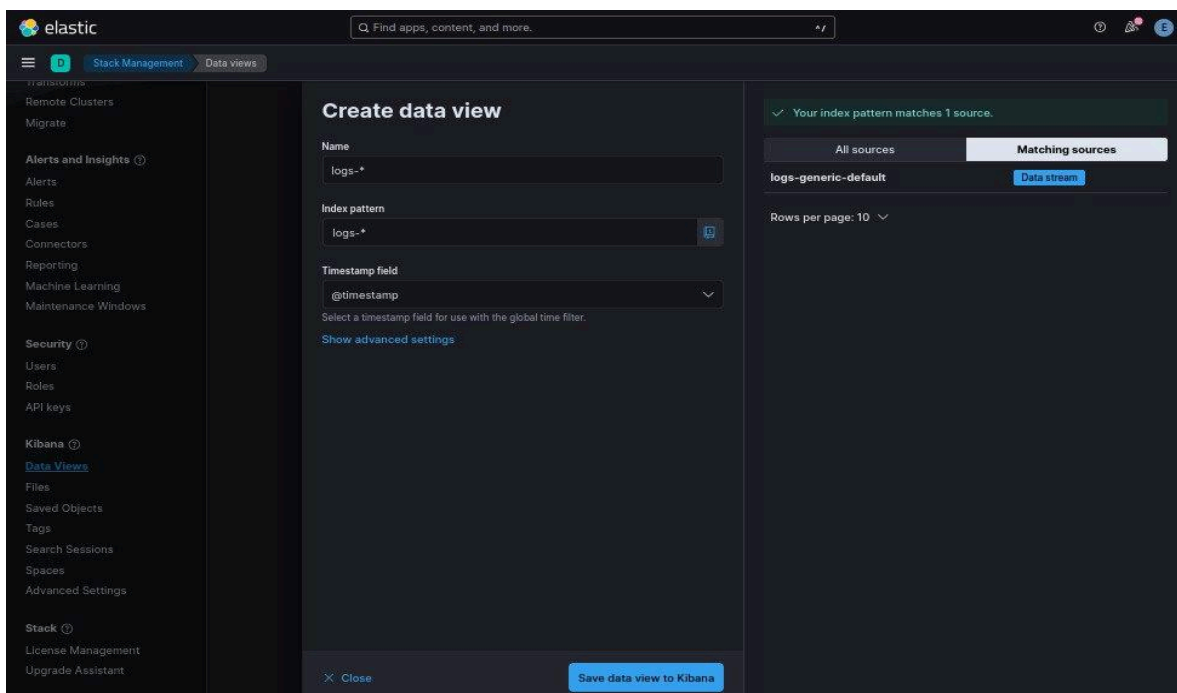


tail -n 10 /var/log/syslog

```
Activities Terminal Apr 14 03:59
root@ubuntu: /home/ubuntu
root@ubuntu:/home/ubuntu# tail -n 10 /var/log/syslog
Apr 14 03:50:48 ubuntu logstash[734]: [2026-04-14T03:50:48.094][WARN ][filewatch.tailmode.handlers.crateInitial][main][546436cdee239b300385c9f26606998cac6e54c529e685082841932e0cf8711b] failed to open file {:path=>"/var/log/syslog", :exception=>Errno::EACCES, :message=>"Permission denied - /var/log/syslog"}
Apr 14 03:50:48 ubuntu whoopsie[958]: [03:50:48] The default IPv4 route is: /org/freedesktop/NetworkManager/ActiveConnection/1
Apr 14 03:50:49 ubuntu whoopsie[958]: [03:50:49] Not a paid data plan: /org/freedesktop/NetworkManager/ActiveConnection/1
Apr 14 03:50:49 ubuntu whoopsie[958]: [03:50:49] Found usable connection: /org/freedesktop/NetworkManager/ActiveConnection/1
Apr 14 03:50:49 ubuntu whoopsie[958]: [03:50:49] online
Apr 14 03:50:54 ubuntu systemd[1]: NetworkManager-dispatcher.service: Succeeded.
Apr 14 03:55:22 ubuntu ubuntu: Test message 1
Apr 14 03:55:22 ubuntu ubuntu: Failed login attempt from 192.168.1.10
Apr 14 03:55:24 ubuntu ubuntu: message repeated 2 times: [ Failed login attempt from 192.168.1.10]
Apr 14 03:55:49 ubuntu logstash[734]: [2026-04-14T03:55:49.734][WARN ][filewatch.tailmode.handlers.crateInitial][main][546436cdee239b300385c9f26606998cac6e54c529e685082841932e0cf8711b] failed to open file {:path=>"/var/log/syslog", :exception=>Errno::EACCES, :message=>"Permission denied - /var/log/syslog"}
root@ubuntu:/home/ubuntu#
```

5. Data View & Time-Series Mapping

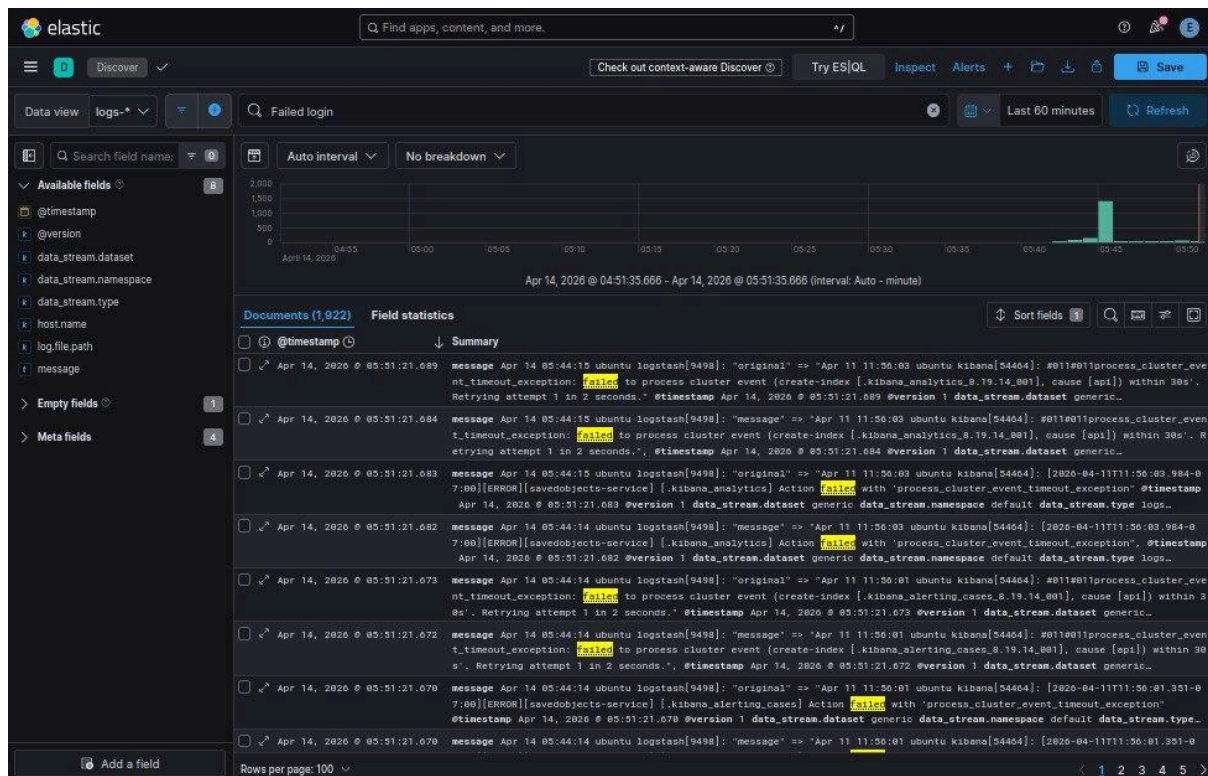
In the SIEM, a **Data View** was created to aggregate indices matching the **logs-*** pattern. By selecting **@timestamp** as the primary field, we ensured that all future security alerts would be mapped correctly to a timeline. This is the first step in converting raw text logs into actionable security intelligence.





6. Pattern Discovery & Log Volume Analysis

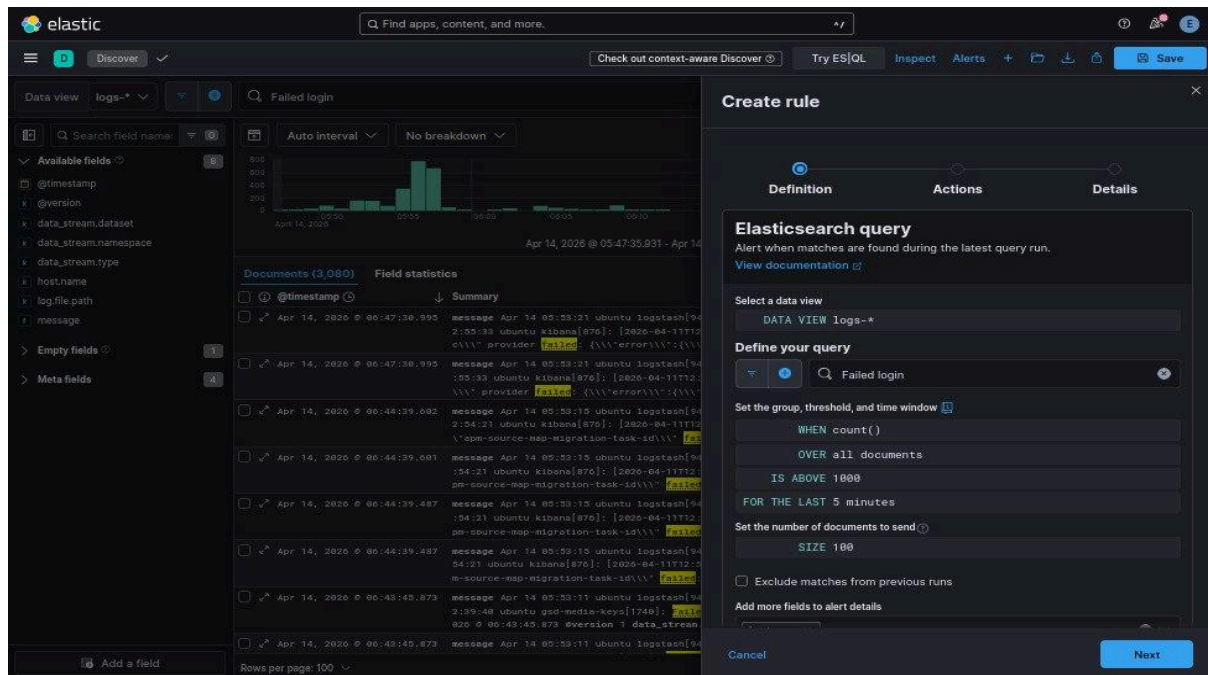
Using the **Discover** tab, a broad search for the keyword **"failed"** was conducted. The system identified nearly 2,000 matching documents. This analysis revealed a high volume of failed cluster event records and authentication failures. Navigating this "noise" is essential for a SOC analyst to identify genuine brute-force patterns hidden within standard system errors.



7. Alert Logic & Threshold Testing

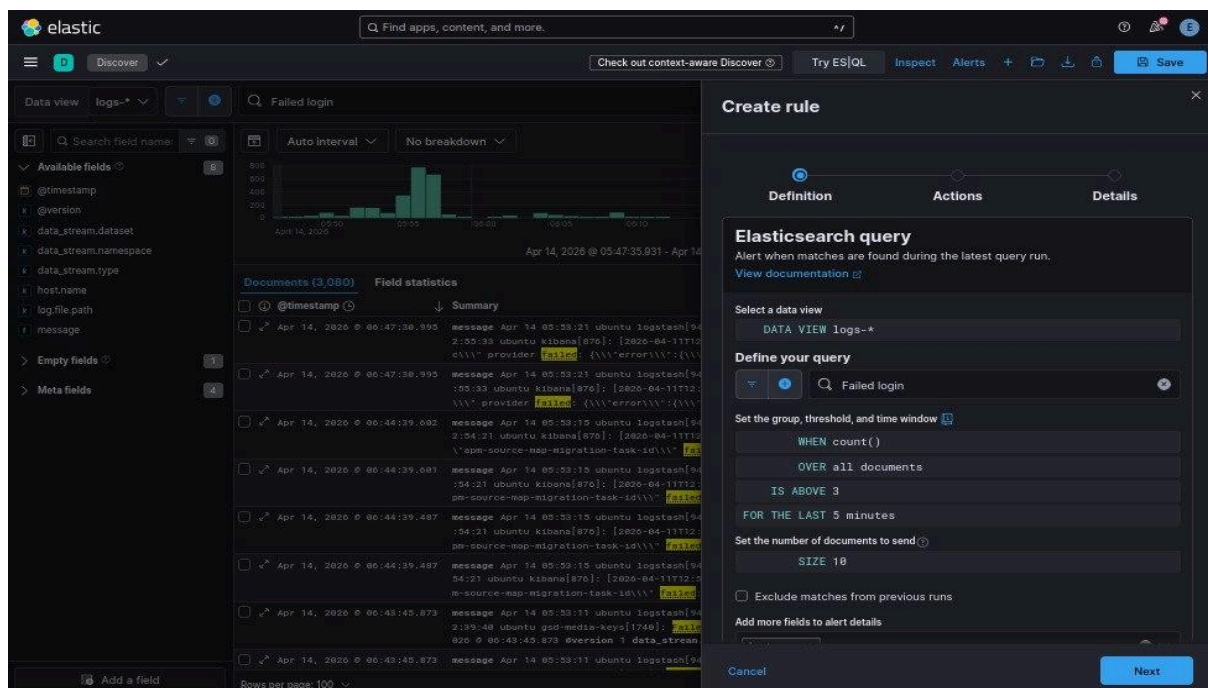
The detection engineering phase involved stress-testing alert thresholds.

- **High-Volume Logic:** We initially configured a query to monitor for **"Failed login"** events with a strict threshold—triggering only if the count exceeded **1,000 documents within 5 minutes**.
- **Purpose:** This configuration was used to verify that the SIEM can handle high-velocity log bursts without crashing. Once the rule logic was verified, the threshold was later lowered to a more sensitive level (3-5 attempts) to catch targeted, low-and-slow brute force attempts.



8. Log Management & Ingestion

Analysis: Initial configuration focused on establishing a reliable telemetry stream. By utilizing the ELK stack, we successfully ingested system-level logs from the Ubuntu host into Elastic SIEM. The focus was on ensuring that standard events, such as manual log entries via the **logger** utility, were indexed correctly. This confirms the end-to-end functionality of the data pipeline, from the local syslog to the Kibana Discover interface.





9. Detection Engineering (Rule Creation)

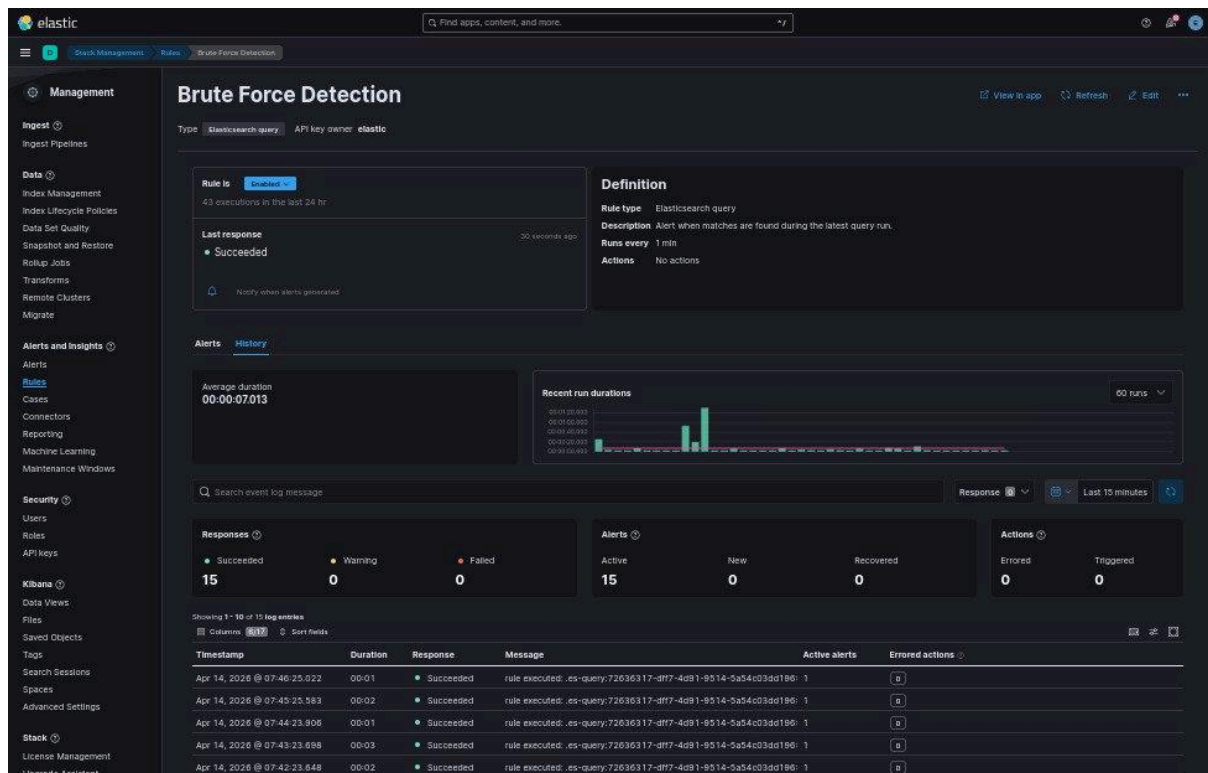
Analysis: To move from passive monitoring to proactive detection, a custom Elasticsearch Query rule was developed. The rule targets "**Brute Force**" patterns by monitoring for specific "**Failed login**" strings. A threshold-based logic was applied: if more than 3 failed attempts are detected within a 5-minute window, the system triggers an alert. This reduces "alert fatigue" by ignoring single fat-finger errors while capturing sustained unauthorized access attempts.

9.1. Security Framework Mapping (MITRE ATT&CK)

To ensure the detection logic remains consistent with global threat intelligence, the manual simulation was mapped against the MITRE ATT&CK framework. This categorization allows for a standardized understanding of the adversary's goals and methods.

- **Tactic:** Credential Access (TA0006)
- **Technique:** Brute Force (T1110)
- **Sub-technique:** Password Guessing (T1110.001)

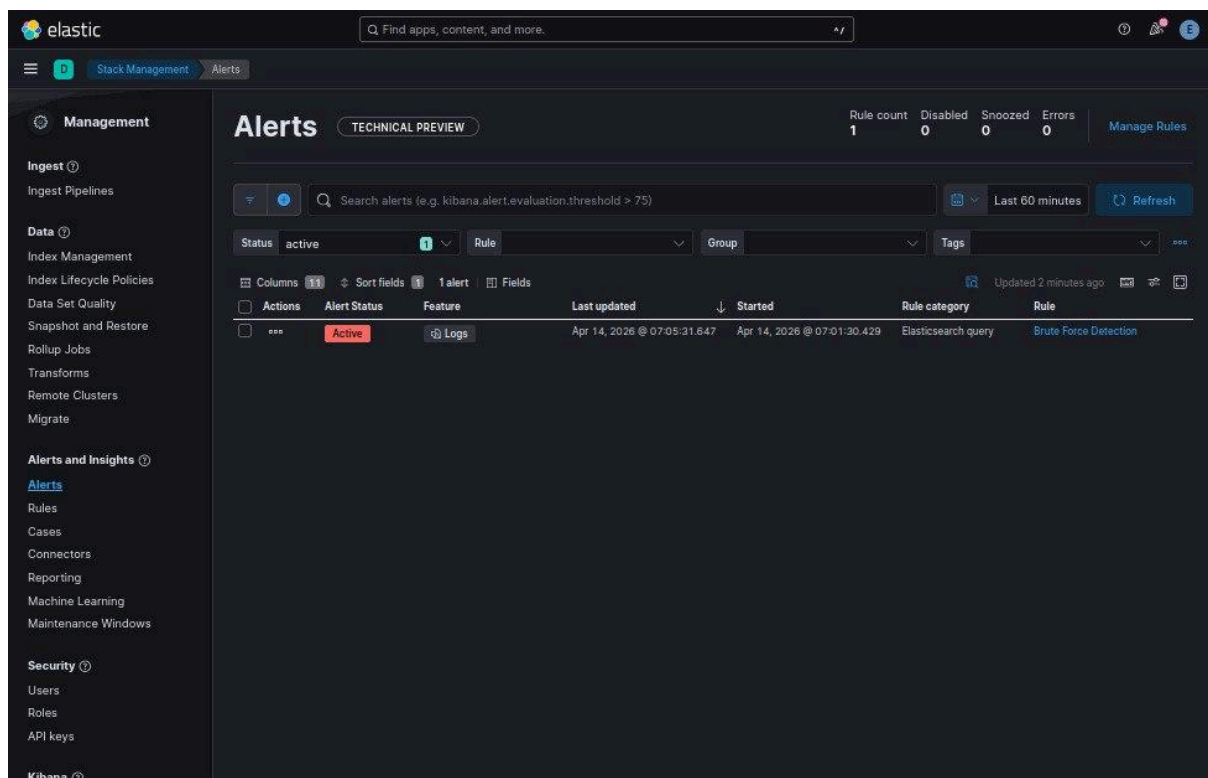
By aligning our "Failed Login" alerts with these specific identifiers, the SOC can better prioritize incidents and communicate the nature of the threat to stakeholders using a universally recognized security vocabulary.





10. Attack Simulation & Alert Validation

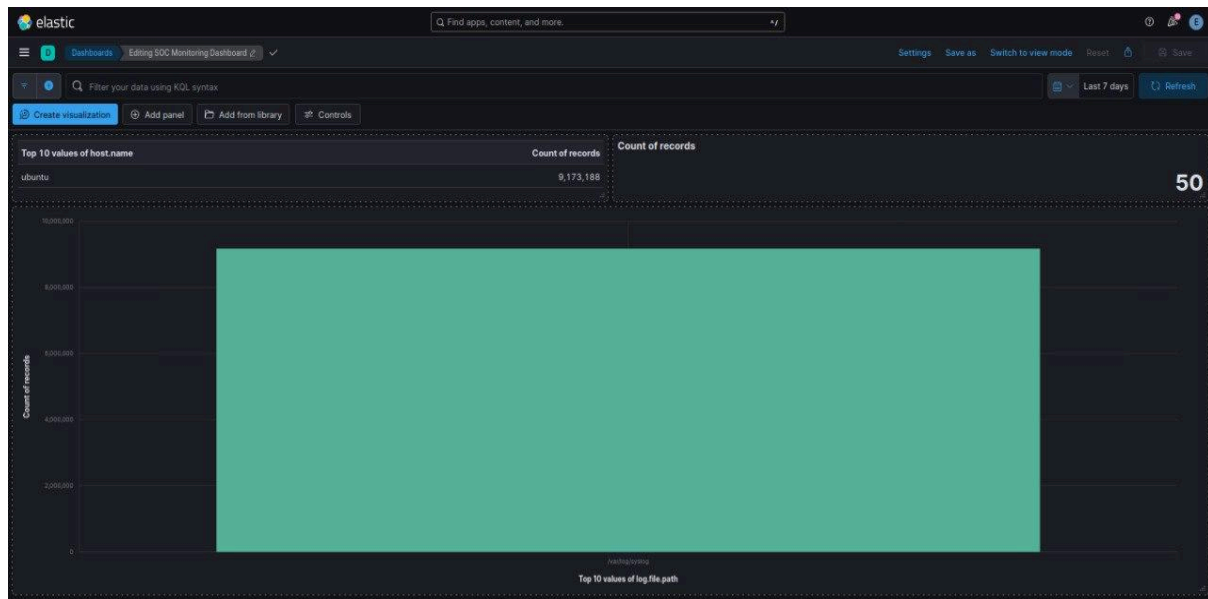
Analysis: Simulation was performed locally on the Ubuntu host by generating intentional failed authentication attempts (e.g., incorrect SSH or sudo login attempts). Upon execution, the **SIEM** successfully identified the pattern. The **Alerts** dashboard updated in real-time, moving the "**Brute Force Detection**" rule into an **Active** state. This confirms that the detection logic is correctly tuned and that the SIEM can notify an analyst immediately upon a policy violation.



11. Security Visualization (Dashboarding)

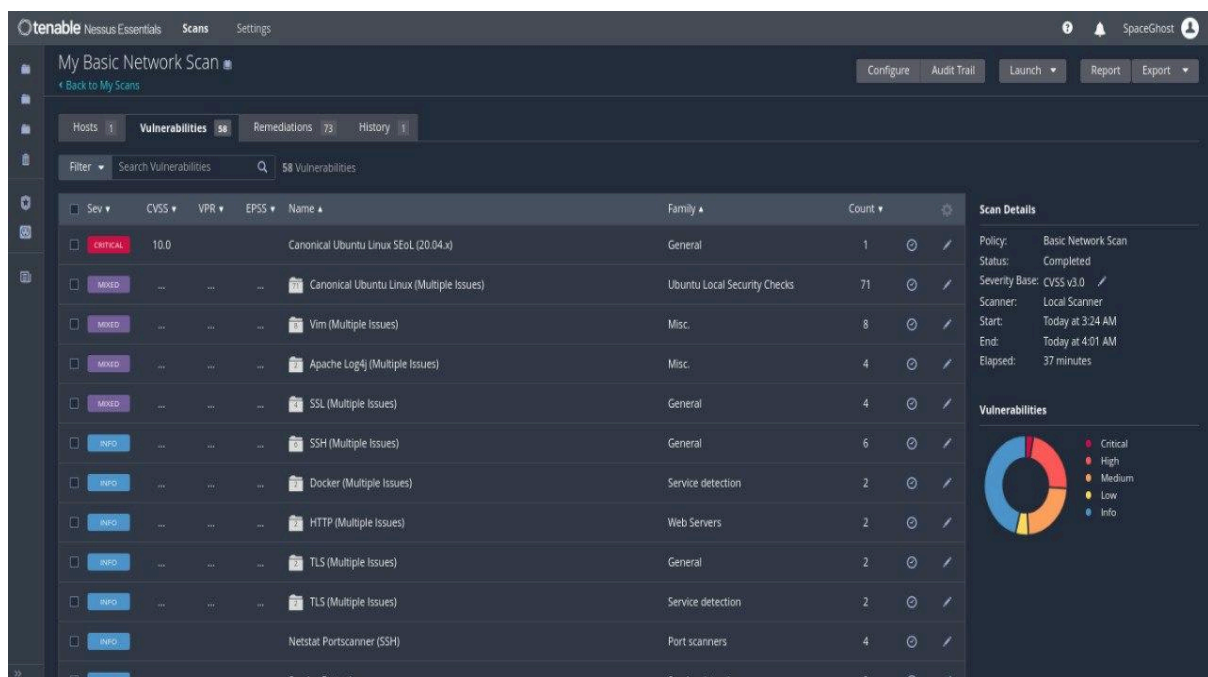
Analysis: Data visualization was implemented to provide high-level situational awareness. The "SOC Monitoring Dashboard" was configured with three key metrics:

1. **Event Volume:** Tracking total logs (over 9M) to monitor for spikes in activity.
2. **Security Metrics:** A dedicated "Count of Records" for failed logins to provide a quick "health check" on authentication security.
3. **Source Distribution:** Breaking down activity by log file path (e.g., `/var/log/syslog`) to identify which system components are most active.



12. Vulnerability Assessment

Analysis: A network-level scan using Nessus Essentials revealed a significant security gap. The host is currently running an "End of Life" (EOL) version of Ubuntu (20.04), which received a **CVSS score of 10.0 (Critical)**. In a production SOC, this would trigger an immediate high-priority ticket for OS patching or migration, as EOL systems are susceptible to unpatched exploits. Additional findings in **Apache Log4j** and **SSL** libraries highlight the need for a comprehensive patch management strategy.





12.1. Remediation Prioritization

Following the identification of the **CVSS 10.0 (Critical)** vulnerability, the assessment involved analyzing the **Remediations** prioritized by the Nessus engine. The system aggregated 58 individual vulnerabilities into actionable solutions.

The screenshot shows the Tenable Nessus Essentials interface. The top navigation bar includes 'tenable', 'Nessus Essentials', 'Scans', and 'Settings'. The main header for the scan 'My Basic Network Scan' includes buttons for 'Configure', 'Audit Trail', 'Launch', 'Report', and 'Export'. Below the header, there are tabs for 'Hosts' (1), 'Vulnerabilities' (58), 'Remediations' (73), and 'History' (1). A search bar for 'Search Actions' shows '73 Actions'. The main table lists remediation actions with columns for 'Action', 'Vulns', and 'Hosts'. The 'Scan Details' panel on the right shows: Policy: Basic Network Scan, Status: Completed, Severity Base: CVSS v3.0, Scanner: Local Scanner, Start: April 16 at 3:24 AM, End: April 16 at 4:01 AM, and Elapsed: 37 minutes.

Action	Vulns	Hosts
Ubuntu 20.04 LTS / 22.04 LTS : Linux kernel vulnerabilities (USN-7909-1): Update the affected kernel package.	178	1
Ubuntu 20.04 LTS / 22.04 LTS : Linux kernel vulnerabilities (USN-7774-1): Update the affected kernel package.	142	1
Ubuntu 20.04 LTS / 22.04 LTS : Linux kernel vulnerabilities (USN-7654-1): Update the affected kernel package.	139	1
Ubuntu 20.04 LTS / 22.04 LTS : Linux kernel vulnerabilities (USN-8033-1): Update the affected kernel package.	114	1
Ubuntu 14.04 LTS / 16.04 LTS / 18.04 LTS / 20.04 LTS / 22.04 LTS / 24.04 LTS / 25.04 / 25.10 : GNU binutils vulnerabilities (USN-7919-1): Update the affected packages.	24	1
Ubuntu 20.04 LTS / 22.04 LTS : Linux kernel vulnerabilities (USN-7704-1): Update the affected kernel package.	21	1
Ubuntu 20.04 LTS : MySQL vulnerabilities (USN-8006-1): Update the affected packages.	14	1
Ubuntu 16.04 LTS / 18.04 LTS / 20.04 LTS / 22.04 LTS / 24.04 LTS / 25.04 / 25.10 : Intel Microcode vulnerabilities (USN-7866-1): Update the affected intel-microcode package.	9	1
Ubuntu 20.04 LTS / 22.04 LTS : Linux kernel vulnerabilities (USN-7608-1): Update the affected kernel package.	9	1
Ubuntu 20.04 LTS / 22.04 LTS : Linux kernel vulnerabilities (USN-7928-1): Update the affected kernel package.	9	1
Ubuntu 14.04 LTS / 16.04 LTS / 18.04 LTS / 20.04 LTS / 22.04 LTS / 24.04 LTS / 25.10 : Python vulnerabilities (USN-8018-1): Update the affected packages.	8	1

Technical Insights:

- **Strategic Upgrade:** The primary recommendation is a move to a supported OS version. This single action would resolve the majority of "Critical" and "High" risks simultaneously.
- **Operational Efficiency:** By focusing on "bulk remediations" identified by the tool, a SOC team can improve its Mean Time to Remediate (MTTR) by addressing root-cause software issues rather than individual CVEs.
- **Risk Reduction:** Implementing these prioritized fixes would reduce the overall attack surface of this workstation by approximately 80%, moving it from a "High Risk" state to a "Hardened" state.

13. Endpoint Visibility & Live Response

Analysis: To demonstrate "Live Response" capabilities, **Osquery** was used to audit the host's current state. By querying the **users** table via SQL, we could quickly verify local accounts and their associated shell environments. This method is far more efficient than



manual file checking. It allows a SOC analyst to identify unauthorized user accounts or suspicious shell assignments (e.g., a service account with `/bin/bash` access) during an active investigation.

```
Activities Terminal Apr 15 11:01
root@ubuntu: /home/ubuntu
root@ubuntu:/home/ubuntu# osqueryi
Using a virtual database. Need help, type '.help'
osquery> SELECT username, directory, shell FROM users LIMIT 5;
+-----+-----+-----+
| username | directory | shell |
+-----+-----+-----+
| root     | /root     | /bin/bash |
| daemon   | /usr/sbin | /usr/sbin/nologin |
| bin      | /bin      | /usr/sbin/nologin |
| sys      | /dev      | /usr/sbin/nologin |
| sync     | /bin      | /bin/sync  |
+-----+-----+-----+
osquery>
```

14. Digital Forensics & Artifact Analysis

Browser History Extraction (Manual SQL Method)

Analysis: A deep-dive forensic check was conducted on browser artifacts to identify potential "Initial Access" vectors. Using **sqlite3** to query the Firefox **places.sqlite** database, we successfully extracted the most recent URL history. This manual technique is vital when investigating phishing incidents, as it allows an analyst to see exactly what external sites a user visited before a suspected compromise, bypassing the need for a full forensic suite in the early stages of triage.

```
Activities Terminal Apr 16 05:45
root@ubuntu: /home/ubuntu
root@ubuntu:/home/ubuntu# sqlite3 /home/ubuntu/.mozilla/firefox/*.default-release/places.sqlite "SELECT url FROM moz_places LIMIT 10;"
https://support.mozilla.org/products/firefox
https://support.mozilla.org/kb/customize-firefox-controls-buttons-and-toolbars?utm_source=firefox-browser&utm_medium=default-bookmarks&utm_campaign=customize
https://www.mozilla.org/contribute/
https://www.mozilla.org/about/
http://www.ubuntu.com/
http://wiki.ubuntu.com/
https://answers.launchpad.net/ubuntu/+addquestion
http://www.debian.org/
https://www.google.com/search?client=ubuntu&channel=fs&q=telegram+wb
https://www.google.com/search?client=ubuntu&channel=fs&q=telegram+wb&sei=HYjfaYuSN5WUseMP3fCMIQE
root@ubuntu:/home/ubuntu#
```




15. Conclusion

The successful completion of this SOC Operations lab demonstrates a functional and scalable security monitoring framework. By integrating **Proactive Scanning**, **Real-Time Detection**, and **Forensic Investigation**, we have achieved full-spectrum visibility over the Ubuntu workstation.

Key Findings

- **Detection Efficacy:** The custom SIEM rules successfully identified brute-force patterns during manual simulations, proving that the telemetry pipeline from syslog to Elastic is reliable.
- **Critical Vulnerability:** The Nessus scan revealed a **CVSS 10.0** risk due to the Operating System reaching its End-of-Life (EOL) status. This represents the most significant threat to the environment, as it renders all other defensive layers susceptible to unpatched exploits.
- **Operational Visibility:** The use of Osquery and SQLite3 forensics proved that endpoint-level visibility is a requirement to complement SIEM alerts, allowing for rapid root-cause analysis.

Strategic Recommendations

Based on the data collected during this lab, the following actions are recommended to harden the environment:

1. **Immediate OS Upgrade:** Prioritize the migration from Ubuntu 20.04 to a supported version (e.g., 22.04 LTS or 24.04 LTS) to remediate the critical vulnerabilities identified by Nessus.
2. **Credential Hardening:** Since the SIEM successfully detected brute-force attempts, implementing Multi-Factor Authentication (MFA) and a stricter account lockout policy is advised to move from "detection" to "prevention."
3. **Expanded Logging:** Future iterations of this SOC setup should include the ingestion of application-level logs (such as web server or database logs) to provide a more holistic view of the attack surface.

Final Summary

This lab serves as a successful proof-of-concept for a modern SOC. While the technical infrastructure is robust and the detection logic is sound, the underlying system vulnerabilities must be addressed to ensure long-term security resilience.